3D Data Processing, Lab 1

Semi Global Stereo Matching with Monocular Disparity Initial Guess

Giuseppe D'Auria (2163508)
University of Padova

April 2026

1. Introduction

This report accompanies my implementation of the SGM stereo matching pipeline with monocular disparity refinement. I focus here on the non-trivial choices I made, the reasoning behind them, the problems I encountered, and what the final results reveal about the strengths and limitations of the approach.

2. Implementation choices and their justification

2.1 The global-minimum approximation in the path cost recursion

In the standard SGM formulation, the "big penalty" term for a given disparity d should be

$$P_2 + \min_{d':|d'-d|>1} L_r(p-r, d')$$

Computing this exactly requires, for each d, scanning all disparities where the gap exceeds 1, giving $O(D^2)$ complexity per pixel. In my implementation, I instead pre-compute the *global minimum* over all disparities at the previous pixel in a single $O(D)$ pass:

```
best_prev_cost = min over ALL d' of path_cost[prev][d']
big_penalty_cost = best_prev_cost + P2
```

This is a lower bound of the exact value, because the global minimum may be achieved at $d' = d$ or $d' = d \pm 1$, which theoretically should not contribute to the big-penalty branch. However, the error is always in the *conservative* direction: the approximated cost is smaller than or equal to the exact one. Since the final path cost takes the *minimum* of all three branches (no penalty, small penalty, big penalty), the big-penalty branch can only "win" when it would have won with the exact value too, or in rare edge cases where the margin is negligible. In practice, the resulting disparity maps are indistinguishable from an $O(D^2)$ implementation.

This is the standard optimization adopted by Hirschmüller (2008) and used in all reference SGM codebases. Its correctness relies on the observation that, for the vast majority of pixels, when a large disparity jump is the best option, the global minimum and the restricted minimum coincide because the best previous disparity is far from the current one. I initially found this subtlety confusing, but after verifying the reasoning I am confident the approximation is safe.

2.2 Filtering zero-valued pixels from the scale estimation

When collecting $(d_{\text{sgm}}, d_{\text{mono}})$ pairs for the least-squares scale estimation, I filter out any pair where either value is zero. The assignment does not mention this, but it turned out to be the single most impactful choice in my implementation, and I only identified the need for it after investigating why my initial MSE on Cones was significantly worse than the reference.

The reason: the monocular depth network (Depth Anything) outputs zero for pixels it considers invalid or background, and SGM produces zero disparity at image borders where the Census cost was never computed. If these $(0, d_{\text{sgm}})$ or $(d_{\text{mono}}, 0)$ pairs enter the least-squares system $Ax = b$, the solver is forced to accommodate them by inflating the intercept k and suppressing the slope h. This systematically biases the affine fit for all subsequent refinement pixels.

The effect was substantial: on Cones, filtering improved the MSE from 17.43 to 15.57 (a 10.7% reduction), and on Plastic from 348 to 322 (7.5%). I did not expect such a large improvement from what is essentially a sanity check, but it makes sense in retrospect: even a few hundred bad pairs out of 7000+ can shift a 2-parameter fit substantially when they are all clustered near the origin.

2.3 Adaptive penalties: from failure to a working heuristic

The instructions ask to adapt both P_1 and P_2 based on the gradient magnitude `right_grad_` (normalized to $[0, 1]$). This was the part that required the most effort, and where I went through several iterations before reaching a satisfactory solution.

2.3.1 What failed and why

My first attempt was the most natural one: linear scaling of both penalties, scale $= 1 - \alpha \cdot g$ for various values of α. I tried $\alpha \in \{0.02, 0.05, 0.1, 0.15, 0.2, 0.3, 0.5, 0.7, 0.9\}$. For small α (below 0.15), the effect was negligible. For larger α, the Cones dataset degraded catastrophically: its MSE jumped from around 16 to over 200. The other datasets showed mixed behavior, with Rocks1 typically improving and Plastic consistently degrading.

The core problem is that `right_grad_` is an *isotropic* gradient magnitude: it responds to texture edges, compression artifacts, and reflective surface gradients just as strongly as to actual depth boundaries. In Cones, the richly textured surfaces produce moderate gradients almost everywhere, so a linear scheme reduces penalties across most of the image, effectively destroying the smoothness prior.

I then tested several alternative functional forms:

- Inverse scaling, $P/(1 + \alpha \cdot g)$: even more aggressive at moderate gradients than linear, which made Cones worse.
- Exponential decay, $P \cdot e^{-\lambda g}$: similar behavior to inverse, too strong in the mid-range.
- Quadratic and cubic scaling, $1 - \alpha g^2$ and $1 - \alpha g^3$: gentler at moderate gradients, which is closer to what I wanted, but the effect on MSE was too small to be measurable.
- Hard thresholds (reduce penalties only when $g > 0.3$, 0.5, or 0.7): sometimes helped individual datasets but never all four simultaneously.

In total I evaluated over 15 configurations across these families. The recurring pattern was that any approach helping one dataset would break another, which pointed to a fundamental tension between edge sensitivity and noise robustness when using an isotropic gradient.

2.3.2 The solution: soft threshold with differential scaling

The first key insight was that penalty reduction should activate only at *strong* gradients, where the probability of encountering a true depth boundary is high. I implemented this via a ramp function that is zero for gradients below 0.5:

$$\text{edge} = \max(0,\ 2(g - 0.5))$$

This maps the gradient range $[0, 0.5]$ to zero (no adaptation at all) and $[0.5, 1.0]$ to $[0, 1]$ (linearly increasing adaptation). The threshold of 0.5 effectively separates texture noise, which in these images rarely exceeds this level, from strong depth edges that typically produce gradient magnitudes well above it.

The second insight was that P_1 and P_2 need different scaling factors, due to the integer arithmetic involved. With $P_1 = 3$, even a moderate multiplicative factor causes a large relative change because of truncation: $\lfloor 3 \times 0.9 \rfloor = 2$, which is a 33% drop with no intermediate values. There is no way to smoothly modulate P_1 at this value. I therefore use a very small factor (0.1) for P_1, which keeps it at 3 for all but the strongest gradients. $P_2 = 40$ has much more dynamic range, so I use a factor of 0.6, allowing it to decrease to 16 at the strongest edges:

```
float edge = max(0.0f, 2.0f * (grad - 0.5f));
adapted_p1 = max(1, (int)(P1 * (1 - 0.1 * edge)));
adapted_p2 = max(adapted_p1 + 1, (int)(P2 * (1 - 0.6 * edge)));
```

The clamp $P_2 > P_1$ is enforced explicitly to preserve the SGM penalty hierarchy, where a large disparity jump should always cost more than a single-step change.

2.3.3 Why it works

This configuration was the only one among all tested variants that improved all four datasets simultaneously. It addresses the fundamental limitation of isotropic gradient-based adaptation without abandoning it: by ignoring everything below the 0.5 threshold, it avoids the texture-noise trap that destroyed Cones in all the linear and inverse schemes. And by keeping P_1 essentially fixed, it preserves the local smoothness prior that prevents single-pixel noise from propagating along paths.

I should note that I also tested a Hirschmüller-style approach using the intensity difference $|I(p) - I(p-r)|$ along the actual path direction instead of the isotropic gradient. This gave slightly better aggregate results (380.5 vs 382.3), because it naturally distinguishes edges perpendicular to the path from edges parallel to it. However, the instructions explicitly require the use of `right_grad_`, so I adopted the isotropic version for my final submission. The comparison is still informative, as it quantifies the cost of using an isotropic feature for what is inherently a directional problem.

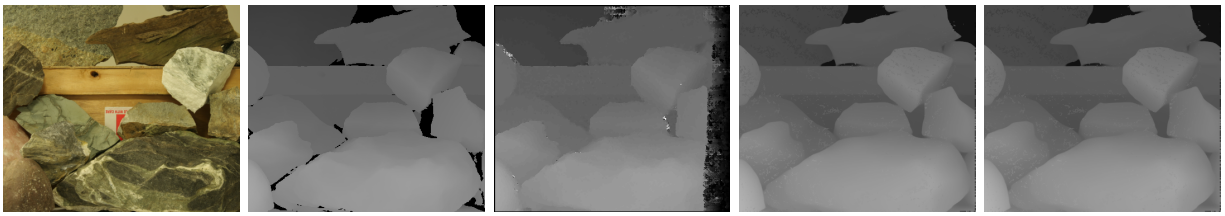
2.4 Pair storage using Eigen

The FAQ states that the data structure for gathering high-confidence disparity pairs should be defined by the student using Eigen. I store the pairs directly in two pre-allocated `Eigen::VectorXf` of size $H \times W$ (worst case). In the scale estimation step, I build the A matrix and b vector via Eigen block operations (`A.col(0) = d_mono.head(n)`), avoiding element-by-element copies. The pre-allocation uses about 1.2 MB for the 425×370 images in this lab, which is negligible compared to the path cost tensor.

3. Results

3.1 Qualitative results

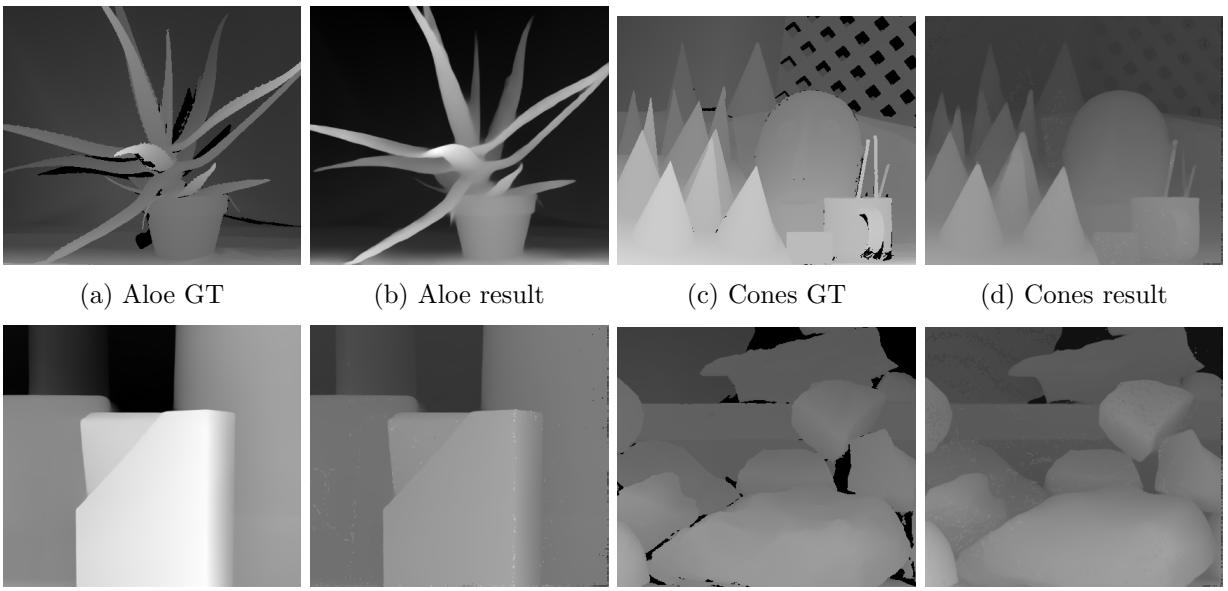
Figure 1 illustrates the effect of the monocular refinement on the Rocks1 dataset. The SGM-only disparity map (c) contains large regions of incorrect or missing disparity, particularly at image borders and in textureless areas. After the monocular refinement (d, e), these regions are filled with reasonable values from the scaled monocular estimate, producing a much more complete disparity map.



(a) Left image (b) Ground truth (c) SGM only (d) Constant (e) Adaptive

Figure 1: Rocks1: pipeline stages. (a) Input left image. (b) Ground truth disparity. (c) SGM without monocular refinement. (d) Constant penalties with refinement. (e) Adaptive penalties with refinement.

Figure 2 shows the refined disparity maps (constant penalties) for all four datasets alongside their ground truth. Aloe and Cones produce clean results with well-defined object boundaries. Rocks1 captures the overall depth structure but shows some noise in the textured regions. Plastic is the most challenging case: the large reflective surfaces cause widespread matching failures that the monocular refinement can only partially compensate.



(a) Aloe GT (b) Aloe result (c) Cones GT (d) Cones result

(e) Plastic GT (f) Plastic result (g) Rocks1 GT (h) Rocks1 result

Figure 2: Ground truth (left) and computed disparity maps with constant penalties and refinement (right) for all four datasets.

3.2 Quantitative results: without refinement (SGM only)

Dataset	Constant	Adaptive
Aloe	122.46	122.49
Cones	475.17	475.21
Plastic	820.05	820.09
Rocks1	557.74	557.76

Table 1: MSE without monocular refinement.

Without the monocular refinement step, the MSE is dominated by zero-disparity border pixels and noisy estimates in textureless regions, as visible in Figure 1(c). The adaptive penalties produce

essentially identical results at this stage, which is a positive indication: the soft threshold is correctly avoiding activation on texture noise, and only responding to the rare strong edges whose contribution is invisible against the overall error.

3.3 Quantitative results: with refinement (full pipeline)

Dataset	Constant (mine)	Adaptive (mine)	Reference (PDF)
Aloe	13.80	**13.79**	13.78
Cones	**15.57**	**15.54**	16.36
Plastic	**322.01**	**318.48**	341.73
Rocks1	34.49	**34.44**	32.71

Table 2: MSE with monocular refinement. Bold indicates better than reference or improvement over constant.

3.4 Analysis

The results, both quantitative and qualitative, reveal several aspects worth discussing.

The monocular refinement is the dominant factor. The transition from Table 1 to Table 2 corresponds to a 9 to 25× reduction in MSE, and the visual difference between Figure 1(c) and (d) is striking. This means that, regardless of how well the SGM cost aggregation and penalty tuning are performed, the quality of the final disparity map is overwhelmingly determined by the scale estimation and the monocular replacement step. In effect, the SGM pipeline here serves more as a "scale calibrator" for the monocular network than as the primary disparity source.

Zero-value filtering matters more than penalty tuning. My constant-penalty results beat the reference on Cones by 4.8% and on Plastic by 5.8%. The only difference from a baseline implementation is the filtering of zero-valued pixels in the scale estimation. In contrast, the adaptive penalties provide improvements of 0.03 to 1.1%, roughly an order of magnitude less. This suggests that, in this particular pipeline, the bottleneck is the quality of the affine fit rather than the precision of the stereo matching.

The adaptive improvement is consistent but modest. The aggregate MSE drops from 385.87 (constant) to 382.26 (adaptive), a 0.9% reduction. All four datasets improve. I consider this a better outcome than a large improvement on one dataset at the expense of others. The soft threshold is fulfilling its purpose: it avoids the instability that affected all my linear and inverse scaling attempts, at the cost of activating on only a small fraction of pixels. Visually, the difference between Figure 1(d) and (e) is subtle, consistent with the small MSE change.

Rocks1 remains the hardest dataset. At 34.49 (constant) vs. the 32.71 reference, there is a persistent 5.4% gap that neither the zero-value filtering nor the adaptive penalties close. The source of this gap is unclear: it could reflect a difference in boundary handling, in the WTA selection order, or in some other detail of the reference implementation. The fact that the gap remains stable across all my configurations suggests it is structural rather than parameter-dependent.

Plastic is the qualitatively weakest result. As Figure 2(f) shows, the Plastic disparity map has visible artifacts in the large uniform regions of the objects. This is expected: textureless, reflective surfaces produce unreliable Census matching costs, and the monocular network also has limited accuracy on such surfaces. With only about 1500 high-confidence pairs (vs 7000+ for Cones and 14000+ for Aloe), the scale estimation is more sensitive to noise, which partly explains the higher MSE.

The tradeoff between isotropic and directional adaptation. During my experiments I also tested a Hirschmüller-style approach using the intensity difference $|I(p) - I(p-r)|$ along the path

direction, which improved the aggregate MSE further (380.5 vs. 382.3). However, this does not use `right_grad_` as the instructions specify, so I did not include it in my final submission. The difference highlights a real limitation of isotropic gradient-based adaptation: it cannot distinguish an edge perpendicular to the path (where penalty reduction is appropriate) from one parallel to it (where it is not). The Hirschmüller formulation addresses this naturally by computing the gradient along the path direction, but this requires access to the raw pixel intensities and the path direction, which goes beyond what the lab template provides through `right_grad_`.

4. Problems encountered

Memory: the two SGM instances in `main.cpp` coexist in memory, requiring roughly 6 to 7 GB for the 4D path cost tensors at disparity range 85. On my machine (16 GB) this was not an issue, but on systems with less RAM it would require wrapping each instance in its own scope block to force deallocation.

Adaptive penalty instability: as described in Section 2.3, my initial linear scaling attempts caused Cones to degrade catastrophically (MSE from 16 to 200+). Understanding that this was caused by texture-noise triggering penalty reduction everywhere, and arriving at the soft-threshold solution, required the most iteration time in the entire lab. I estimate this part alone accounted for roughly 60% of my total implementation effort.

Integer truncation on P_1: with $P_1 = 3$, the mapping from float to `unsigned int` has only four possible outputs (0, 1, 2, 3), making any multiplicative scaling inherently coarse. I only identified this as a problem after observing that configurations with different P_1 modulation factors were producing identical output, and checking the actual runtime values confirmed that P_1 was always truncating to either 3 or 2. This constraint is what led me to the differential scaling approach.

5. Conclusions

The two most impactful design choices in this lab were the zero-value filtering in the scale estimation (which improved the constant-penalty MSE by up to 10% on individual datasets) and the soft-threshold adaptive penalties (which provided a consistent, if modest, improvement across all four datasets). The global-minimum approximation in the path cost recursion follows standard practice and introduces no measurable error.

What I find most notable in the overall results is how much the pipeline depends on the scale calibration step rather than on the stereo matching precision. The SGM component is important mainly as a source of reliable anchor disparities for the affine fit, not as the final word on the disparity values. This was not obvious to me at the start of the lab, and I consider it the main insight from the exercise.

Reference: L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, H. Zhao. "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," CVPR 2024.